

FastAPI Learning Notes

Personal Task Manager project handbook: how the backend, database, authentication, workspaces, and frontend connection fit together.

Built from your project conversation: FastAPI + SQLAlchemy + SQLite + Pydantic + JWT authentication + React integration.

What you built	Main lessons
Authentication	Register, login, password hashing, OAuth2 form login, JWT creation, token verification, protected routes.
Tasks	CRUD endpoints, personal tasks, workspace tasks, owner checks, status codes, validation.
Workspaces	Many-to-many members, workspace-specific tasks, authorization through membership.
Frontend connection	React services, fetch, Authorization: Bearer token, CORS, protected dashboard.

1. Big Picture

The Request Cycle

Your application is a full-stack request-and-response system. The browser does not talk directly to SQLite. React sends an HTTP request to FastAPI, FastAPI validates and authenticates it, SQLAlchemy talks to SQLite, then FastAPI returns JSON back to React.

```
User clicks button
-> React event handler
-> service function with fetch()
-> FastAPI route
-> Depends() dependencies
-> SQLAlchemy Session
-> SQLite database
-> JSON response
-> React state updates
-> UI re-renders
```

2. FastAPI Fundamentals

FastAPI app

`app = FastAPI()` creates the web application object. Routes are attached to this app directly or through included routers.

HTTP methods

GET reads, POST creates, PUT/PATCH updates, DELETE removes. This matches CRUD: Create, Read, Update, Delete.

Swagger/OpenAPI

FastAPI automatically creates API docs at `/docs`. If the server is not running, the browser cannot reach Swagger. If a router is not included, its endpoints will not appear there.

```
from fastapi import FastAPI
from routers import auth, tasks, workspaces

app = FastAPI(title="Personal Task Manager API")

app.include_router(auth.router, prefix="/auth", tags=["auth"])
app.include_router(tasks.router, prefix="/tasks", tags=["tasks"])
app.include_router(workspaces.router, prefix="/workspaces", tags=["workspaces"])
```

3. APIRouter And Route Structure

Why routers exist

APIRouter lets you split endpoints by feature: auth routes in `auth.py`, task routes in `tasks.py`, workspace routes in `workspaces.py`. `main.py` stays small.

Prefix behavior

If `main.py` includes `tasks.router` with `prefix='/tasks'` and the router has `@router.get('/personal')`, the final URL is `GET /tasks/personal`.

Common route mistake

A route function can be correct but invisible if the router is not imported and included in `main.py`. Another common issue is frontend calling a URL that does not match the prefix/path combination.

```
from fastapi import APIRouter

router = APIRouter()

@router.get("/personal")
```

```
def get_personal_tasks():
    return []

# Included with prefix="/tasks"
# Final endpoint: GET /tasks/personal
```

4. Depends And Dependency Injection

Depends()

Depends tells FastAPI: before running this route, run another function and inject its return value.

Database dependency

get_db creates a SQLAlchemy session for the request and closes it afterward.

Auth dependency

get_current_user reads the bearer token, verifies it, loads the user, and gives the route the current user object.

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@router.get("/personal")
def list_tasks(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    return db.query(Task).filter(Task.owner_id == current_user.id).all()
```

5. SQLAlchemy Engine, Session, Base

Engine

The engine is SQLAlchemy's connection manager for the database. With SQLite it points to a file such as `sqlite:///./app.db`.

SessionLocal

A session is the unit of work for database operations. You query, add, commit, refresh, and delete through the session.

Base

Models inherit from Base so SQLAlchemy can map Python classes to database tables.

create_all

Base.metadata.create_all(bind=engine) creates missing tables from your model definitions during development.

```
DATABASE_URL = "sqlite:///./app.db"

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False},
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

class Base(DeclarativeBase):
    pass
```

6. Models, Columns, Keys, Relationships

Model

A model class represents a database table. Its attributes represent columns and relationships.

Primary key

`id = Column(Integer, primary_key=True)` uniquely identifies each row.

Foreign key

`ForeignKey('users.id')` stores a database-level link from one table to another.

relationship()

`relationship()` gives Python navigation between linked models, such as `user.tasks` or `task.owner`.

```
class Task(Base):
    __tablename__ = "tasks"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String, nullable=False)
    owner_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    workspace_id = Column(Integer, ForeignKey("workspaces.id"), nullable=True)

    owner = relationship("User", back_populates="tasks")
    workspace = relationship("Workspace", back_populates="tasks")
```

7. Pydantic Schemas, Field, from_attributes

BaseModel

Pydantic schemas describe the shape of incoming requests and outgoing responses.

Field

Field adds validation rules such as required values, `min_length`, `max_length`, `ge`, and `le`.

Request vs response schemas

Create schemas can include passwords; response schemas should hide secrets like `hashed_password`.

from_attributes

This lets Pydantic build a response model from SQLAlchemy object attributes.

```
class TaskCreate(BaseModel):
    title: str = Field(..., min_length=1, max_length=100)
    description: str | None = None
    priority: int = Field(1, ge=1, le=5)
    workspace_id: int | None = None

class TaskResponse(BaseModel):
    id: int
    title: str
    priority: int

    model_config = ConfigDict(from_attributes=True)
```

8. CRUD Endpoints

Create

Build a model object from validated schema data, attach ownership fields, add it, commit it, refresh it, return it.

Read

Query records that belong to the current user or a workspace where the user is a member.

Update

Load the existing row first, verify permission, change allowed fields, commit.

Delete

Load, verify permission, delete, commit, then return a success message or 204 response.

```
@router.post("/personal", response_model=TaskResponse, status_code=201)
def create_personal_task(
    payload: TaskCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    task = Task(**payload.model_dump(exclude={"workspace_id"}), owner_id=current_user.id)
    db.add(task)
    db.commit()
    db.refresh(task)
    return task
```

9. Authentication: Hashing, OAuth2, JWT

Password hashing

Never store plain passwords. Hash them using bcrypt/passlib, then verify login attempts against the stored hash.

OAuth2PasswordRequestForm

FastAPI's form helper expects username and password form fields. Many apps put the email into the username field.

JWT

A JWT has header.payload.signature. The payload stores claims such as sub and exp. The signature proves the token was created with your SECRET_KEY.

Bearer token

React stores the access token and sends Authorization: Bearer . FastAPI extracts it using OAuth2PasswordBearer.

```
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")

def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db),
):
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    email = payload.get("sub")
    user = db.query(User).filter(User.email == email).first()
    if not user:
        raise HTTPException(status_code=401, detail="Invalid token")
    return user
```

10. Workspaces And Many-to-Many Members

Why many-to-many

A user can join many workspaces, and a workspace can contain many users. That requires an association table.

Association table

The join table stores pairs of user_id and workspace_id. Each pair means this user is a member of that workspace.

Workspace authorization

Before reading or changing workspace tasks, check that current_user belongs to that workspace.

```
user_workspace = Table(
    "user_workspace",
    Base.metadata,
```

```

        Column("user_id", Integer, ForeignKey("users.id"), primary_key=True),
        Column("workspace_id", Integer, ForeignKey("workspaces.id"), primary_key=True),
    )

    class User(Base):
        __tablename__ = "users"
        id = Column(Integer, primary_key=True)
        workspaces = relationship("Workspace", secondary=user_workspace, back_populates="members")

    class Workspace(Base):
        __tablename__ = "workspaces"
        id = Column(Integer, primary_key=True)
        members = relationship("User", secondary=user_workspace, back_populates="workspaces")

```

11. Personal Tasks Vs Workspace Tasks

Personal task

A personal task belongs to a user and normally has workspace_id = None.

Workspace task

A workspace task belongs to a workspace and can still track who created or owns it.

Common bug

If workspace_id is nullable=False, personal task creation fails because personal tasks do not have a workspace. Make workspace_id nullable=True or split models/tables.

Permission rule

Personal tasks are filtered by owner_id. Workspace tasks are filtered by workspace_id plus membership.

```

# Personal task query
db.query(Task).filter(
    Task.owner_id == current_user.id,
    Task.workspace_id.is_(None),
).all()

# Workspace task query
workspace = db.query(Workspace).filter(Workspace.id == workspace_id).first()
if current_user not in workspace.members:
    raise HTTPException(status_code=403, detail="Not a workspace member")

```

12. HTTP Status Codes And Errors

200 OK

Successful read or update.

201 Created

Successful create.

204 No Content

Successful delete with no response body.

400 Bad Request

The request is structurally understandable but logically wrong.

401 Unauthorized

Missing or invalid authentication token.

403 Forbidden

Authenticated, but not allowed to access that resource.

404 Not Found

The requested user, task, or workspace does not exist.

422 Validation Error

Pydantic/FastAPI rejected the request body, path, query, or form data.

```
raise HTTPException(status_code=404, detail="Task not found")
raise HTTPException(status_code=403, detail="Not allowed to edit this task")
```

13. Environment Variables, SECRET_KEY, CORS

Environment variables

Values like DATABASE_URL, SECRET_KEY, and frontend API URLs should live in .env rather than being hardcoded.

SECRET_KEY

The SECRET_KEY signs JWTs. If it changes, old tokens stop verifying. If it leaks, attackers may forge tokens.

CORS

React and FastAPI usually run on different origins during development, such as localhost:5173 and 127.0.0.1:8000. CORS tells the browser which frontend origins may call the API.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

14. Frontend And Backend Connection

Service functions

React service files hide fetch details from components. authService handles login/register/token storage; taskService handles task API calls.

ProtectedRoute

The frontend checks whether a token exists before showing the dashboard. The backend still remains the real security boundary.

Authorization header

Every protected API call must include Authorization: Bearer . Without it, get_current_user cannot know who is calling.

```
export async function getPersonalTasks() {
  const token = localStorage.getItem("access_token");
  const response = await fetch("http://127.0.0.1:8000/tasks/personal", {
    headers: { Authorization: `Bearer ${token}` },
  });
  if (!response.ok) throw new Error("Could not load tasks");
  return response.json();
}
```

15. Common Mistakes From This Project

NOT NULL task id

If a primary key id is not auto-generated or the table schema is out of sync, inserts can fail. Check primary_key=True, autoincrement behavior, migrations/table recreation, and whether the database file is stale.

Workspace-required personal task issue

Personal tasks broke when the task model required `workspace_id`. Personal tasks need `workspace_id` nullable or a separate personal-task model.

Swagger not loading

Usually the server is not running, running on a different port, or has an import error during startup.

Endpoint missing in docs

The router was created but not included, or the prefix/path did not match what you expected.

OAuth2 form confusion

`OAuth2PasswordRequestForm` sends form fields, not JSON. If logging in with email, the email may arrive as `form_data.username`.

Frontend URL mismatch

React might call `/tasks/create` while FastAPI exposes `/tasks/personal`. Match method, URL, body, and headers exactly.

```
# Good debugging order
1. Is the FastAPI server running?
2. Does /docs show the endpoint?
3. Does Swagger request work?
4. Does React call the same method and URL?
5. Is the token being sent?
6. Does the database schema match the model?
```

16. Practical Request Flows

Register flow

React register form

- > POST /auth/register with username, email, password
- > Pydantic validates email and fields
- > FastAPI checks if user already exists
- > password is hashed
- > User row is saved
- > response returns safe user fields, not hashed_password

Login flow

React login form

- > POST /auth/login as form data
- > backend finds user by email/username
- > verifies password against hashed password
- > creates JWT with sub and exp
- > React stores access_token
- > frontend navigates to dashboard

Protected task flow

Dashboard opens

- > useEffect calls getPersonalTasks()
- > fetch sends Authorization: Bearer <token>
- > OAuth2PasswordBearer extracts token
- > get_current_user verifies JWT
- > route queries tasks for current_user.id
- > JSON list returns
- > setTasks(data)
- > UI renders real tasks

17. Testing Checklist Before Real-Time Collaboration

- Run the backend and confirm /docs opens without import errors.
- Register a new user and confirm duplicate emails are rejected.
- Log in with valid and invalid credentials.
- Confirm the returned JWT is stored by the frontend and sent on protected requests.
- Visit /dashboard while logged out and confirm the frontend redirects to /login.
- Create, list, update, and delete personal tasks.
- Confirm one user cannot see or edit another user's personal tasks.
- Create a workspace, add/list members, and create workspace tasks.
- Confirm non-members receive 403 for workspace resources.
- Test empty titles, invalid priority values, missing token, expired token, missing task, and missing workspace.
- Stop hardcoding API URLs; move frontend API base URL and backend secrets to environment variables.
- Add loading and error states in React for every network request.
- Write a README with setup, env vars, run commands, and endpoint overview.
- Commit the stable MVP before adding WebSockets or real-time collaboration.

18. Architecture Cheat Sheet

Piece	Job	Remember
FastAPI	Receives HTTP requests and returns JSON.	Routes are the public API surface.
APIRouter	Groups endpoints by feature.	include_router activates it.

Piece	Job	Remember
Depends	Runs shared logic before route logic.	Great for db sessions and auth.
SQLAlchemy Engine	Connects SQLAlchemy to SQLite.	One engine per database config.
Session	Per-request database work area.	Add, query, commit, refresh, delete.
Base	Parent for mapped models.	Models become tables.
Pydantic	Validates input and shapes output.	Use schemas, not raw dictionaries everywhere.
JWT	Portable proof of login.	Signed with SECRET_KEY; send as Bearer token.
CORS	Browser permission for cross-origin API calls.	Needed for React dev server to call FastAPI.
React services	Central place for fetch calls.	Keep components focused on UI/state.

19. Final Mental Model

A route is not just a function. In your project, a route is the meeting point of validation, authentication, authorization, database work, and frontend state. Pydantic decides whether the incoming data is valid. Depends supplies the database and current user. SQLAlchemy turns Python object operations into SQL. JWT proves who the user is. React uses the JSON response to update the screen.

That is the core pattern behind many production APIs. Your Personal Task Manager taught it through real pieces: users, tasks, workspaces, protected pages, tokens, relationships, errors, and debugging.